

Major Project Report

on

**Cognitive OS: AI-Powered Cognitive Load
Management for Software Engineers**

Submitted by

Aryan Jaiswal 22BCS018

Pranay Ch 22BCS088

Vinay Jain 22BCS137

Under the guidance of

Malay Kumar

Assistant Professor, Computer Science & Engineering



**INDIAN INSTITUTE OF
INFORMATION
TECHNOLOGY**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY DHARWAD**

April, 2026

Certificate

This is to certify that the project, entitled **Cognitive OS: AI-Powered Cognitive Load Management for Software Engineers**, is a bonafide record of the Major Project coursework presented by the students whose names are given below during 2024–2026 in partial fulfilment of the requirements of the degree of Bachelor of Technology in Computer Science and Engineering.

Roll No	Names of Students
---------	-------------------

22BCS018	Aryan Jaiswal
22BCS088	Pranay Ch
22BCS137	Vinay Jain

Malay Kumar
(Project Supervisor)

Abstract

Software developer burnout is a growing crisis in the technology industry, with global losses estimated at \$3 trillion annually due to disengagement and cognitive overload [6]. Traditional approaches to measuring cognitive load rely on subjective surveys or intrusive physiological sensors, both of which are impractical for continuous, at-scale deployment. This project presents **Cognitive OS**, a web-based platform that automatically infers real-time cognitive load from GitHub activity data using seven research-backed mathematical formulas. The system computes a composite Cognitive Load Index (CLI) on a 0–100 scale, performs anomaly detection via statistical z-scores, predicts burnout risk, and quantifies productivity gains from reduced context switching. We collected data from **51 public GitHub developers over 30+ days**, analyzing over 15,000 commits, 2,000 pull requests, and 5,000 issues. Our CLI metric achieves a **Pearson correlation of 0.84** with self-reported stress levels from a 10-participant validation study. The system projects a median time savings of **23 hours per developer per month** and an average productivity improvement of **15.9%**. Cognitive OS is built with Next.js 16, React 19, PostgreSQL, Redis, and Azure OpenAI, and includes AI-powered agents for focus protection, task planning, and interrupt guarding.

Contents

Abstract	i
List of Figures	iv
List of Tables	v
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope and Contributions	2
1.5 Report Organization	3
2 Related Work	4
2.1 Cognitive Load Theory	4
2.2 Workload Assessment Instruments	4
2.3 Context Switching and Interruptions	4
2.4 Developer Productivity Frameworks	5
2.5 Burnout in Software Engineering	5
2.6 Code Complexity Metrics	5
2.7 Gap Analysis	6
3 Data and Methods	7
3.1 System Architecture	7
3.2 Technology Stack	8
3.3 Data Collection	8
3.4 Data Model	9
3.5 Mathematical Formulations	9
3.5.1 Formula 1: Cognitive Load Index (CLI)	9
3.5.2 Formula 2: Adaptive Weight Learning	10
3.5.3 Formula 3: Historical Blending with Exponential Decay	10

3.5.4	Formula 4: Anomaly Detection via Z-Score	11
3.5.5	Formula 5: Context Switch Cost Model	11
3.5.6	Formula 6: Burnout Risk Prediction	11
3.5.7	Formula 7: Productivity Gain Measurement	12
3.6	AI Agent Architecture	12
3.7	Caching Strategy	12
3.8	API Surface	13
4	Results and Discussions	14
4.1	Cognitive Load Distribution	14
4.2	Context Switches vs. Cognitive Load	15
4.3	Burnout Risk Distribution	16
4.4	Projected Time Savings by Activity Category	16
4.5	Productivity Gain vs. Baseline Context Switches	17
4.6	Monthly Cost Impact	18
4.7	7-Day CLI Trend with Anomaly Detection	18
4.8	Focus Ratio vs. Burnout Risk	19
4.9	Comparative Summary	20
4.10	Discussion	20
5	Conclusion	22
5.1	Summary	22
5.2	Limitations	22
5.3	Future Work	22
	References	24

List of Figures

1	System architecture of Cognitive OS showing data flow between layers.	7
2	Distribution of mean Cognitive Load Index (CLI) scores across 51 developers. The distribution is approximately normal with a mean of 38.7 and standard deviation of 17.2.	14
3	Scatter plot of average daily context switches vs. mean CLI score ($n = 51$). The Pearson correlation is $r = 0.84$ ($p < 0.001$), indicating a strong positive relationship.	15
4	Burnout risk distribution across 51 tracked developers. 25.5% ($n = 13$) are at High or Critical risk.	16
5	Projected monthly time savings by developer activity category. High-activity developers save over 38 hours per month.	17
6	Projected productivity gain as a function of baseline context switches, comparing the full AI agent suite (35% reduction) vs. dashboard-only mode (20% reduction).	17
7	Monthly cost of context switching per developer, before and after Cognitive OS intervention (35% reduction in switches).	18
8	Example 7-day CLI trend for a developer. Thursday shows a severe anomaly ($z = 2.7$) triggered by a production incident.	18
9	Scatter plot of focus ratio vs. burnout risk score ($n = 51$). Strong negative correlation ($r = -0.91$) confirms that developers with more focused time have lower burnout risk.	19

List of Tables

1	Comparison of Existing Approaches vs. Cognitive OS	6
2	Technology Stack Summary	8
3	Database Schema Overview	9
4	CLI Weight Parameters	10
5	API Endpoint Summary	13
6	Projected Time and Cost Savings by Developer Category	16
7	Aggregate Statistics from 51-Developer Study	20

1 Introduction

1.1 Background and Motivation

The global software industry employs over 28 million developers, each navigating an increasingly complex landscape of tools, codebases, and collaborative workflows. A 2023 Gallup report estimates that employee disengagement and burnout cost the global economy approximately \$3 trillion per year [6]. For software engineers specifically, the problem is compounded by the inherently cognitive nature of their work: holding complex mental models, reasoning about distributed systems, and constantly switching between disparate tasks.

Cognitive Load Theory (CLT), introduced by Sweller [18], provides a theoretical framework for understanding the limits of working memory. When cognitive load exceeds capacity, performance degrades, error rates increase, and burnout risk escalates. Despite decades of research in educational psychology, the application of CLT to software engineering workflows has remained largely theoretical [19].

The seminal work by Mark, Gudith, and Klocke [10] demonstrated that it takes an average of **23 minutes and 15 seconds** to fully refocus after an interruption. For a developer who switches context even three times per day, this translates to approximately 70 minutes of lost productive time daily. Leroy [9] further established the concept of *attention residue*—the phenomenon where thoughts from a prior task persist and diminish performance on the current task.

Existing approaches to measuring developer cognitive load fall into three categories: (1) self-report surveys such as the NASA Task Load Index (NASA-TLX) [8], which are subjective and cannot be administered continuously; (2) physiological sensors measuring galvanic skin response, heart rate variability, or EEG [5, 14], which are intrusive and impractical at scale; and (3) proxy metrics derived from development activity [13], which lack formal cognitive load models. None of these approaches provide real-time, non-intrusive, continuous cognitive load measurement from work artifacts alone.

1.2 Problem Statement

There exists no production-grade system that can:

1. Continuously infer a developer’s cognitive load from their work activity without surveys or sensors;
2. Provide actionable, real-time interventions (break recommendations, task deferral, interrupt guarding);
3. Quantify the economic impact of cognitive overload and context switching in monetary terms;
4. Predict burnout risk before it manifests in attrition or health issues.

1.3 Objectives

The objectives of this project are:

1. To design and implement a composite Cognitive Load Index (CLI) based on six sub-factors derived from GitHub activity data.
2. To develop seven mathematical formulas grounded in established cognitive science literature for load computation, adaptive personalization, historical smoothing, anomaly detection, context switch costing, burnout prediction, and productivity measurement.
3. To build a full-stack web application that computes these metrics in real time and presents them through an interactive dashboard.
4. To deploy three AI-powered agents (Focus Agent, Planning Agent, Interrupt Guard) that provide autonomous, context-aware recommendations.
5. To validate the system against self-reported stress data and demonstrate correlation with established burnout indicators.

1.4 Scope and Contributions

This project makes the following contributions:

- A novel composite metric (CLI) that fuses six cognitive load factors into a single 0–100 score with adaptive personalization.
- An empirical dataset of 51 developers tracked over 30+ days with full breakdown data.
- A complete open-source platform with 19 REST API endpoints, real-time dashboards, and AI-powered agents.

- A validated correlation ($r = 0.84$) between algorithmic cognitive load scores and self-reported stress.
- An economic model that quantifies developer productivity loss from context switching at \$101/hour.

1.5 Report Organization

The remainder of this report is organized as follows. Section 2 reviews related work in cognitive load theory, developer productivity, and burnout prediction. Section 3 describes the system architecture, mathematical formulations, and implementation details. Section 4 presents experimental results, statistical analysis, and comparative evaluation. Section 5 concludes with limitations and directions for future work.

2 Related Work

2.1 Cognitive Load Theory

Cognitive Load Theory (CLT), proposed by Sweller [18], distinguishes three types of cognitive load: *intrinsic* (inherent task difficulty), *extraneous* (imposed by poor design or irrelevant information), and *germane* (load devoted to constructing schemas). In 2019, Sweller, van Merriënboer, and Paas revisited the theory and acknowledged its applicability beyond education [19]. Our work maps these concepts to software engineering: intrinsic load corresponds to task complexity, extraneous load to context switching and tool overhead, and germane load to deep-focus programming sessions.

2.2 Workload Assessment Instruments

The NASA Task Load Index (NASA-TLX) by Hart and Staveland [8] is the gold standard for subjective workload assessment. It measures six dimensions: mental demand, physical demand, temporal demand, performance, effort, and frustration. While highly validated, NASA-TLX requires manual administration and is impractical for continuous monitoring. Fritz et al. [5] explored psycho-physiological measures (EDA, EEG, pupil dilation) for assessing task difficulty in software development, achieving moderate classification accuracy but requiring specialized hardware.

2.3 Context Switching and Interruptions

Mark, Gudith, and Klocke [10] conducted the landmark study on interruption costs, finding that workers require an average of 23 minutes and 15 seconds to return to an interrupted task. González and Mark [7] further showed that workers spend an average of only 3 minutes on any single event before switching. Parnin and Rugaber [16] studied resumption strategies specific to programming tasks, finding that developers who were interrupted during method editing required up to 15 minutes to resume. Leroy [9] formalized *attention residue* as the cognitive cost of incomplete task switching.

2.4 Developer Productivity Frameworks

The SPACE framework by Forsgren et al. [4] proposes five dimensions of developer productivity: Satisfaction, Performance, Activity, Communication, and Efficiency. The DevEx framework by Noda et al. [15] refines this with three dimensions: flow state, feedback loops, and cognitive load. Forsgren, Humble, and Kim [3] in *Accelerate* established the four DORA metrics (deployment frequency, lead time, change failure rate, MTTR) as the standard for software delivery performance. Meyer et al. [13] studied developers’ daily routines and found that perceived productivity correlates strongly with the ability to maintain uninterrupted focus.

2.5 Burnout in Software Engineering

The Maslach Burnout Inventory—General Survey (MBI-GS) [11] measures three dimensions of burnout: emotional exhaustion, depersonalization, and reduced personal accomplishment. Müller and Fritz [14] explored the relationship between developer emotions and progress, finding that “stuck” states correlate with negative affect and reduced output. Storey et al. [17] documented how tool fragmentation contributes to developer frustration.

2.6 Code Complexity Metrics

McCabe’s Cyclomatic Complexity [12] measures the number of independent paths through source code and remains widely used. Campbell [2] introduced Cognitive Complexity as an improved metric that penalizes nesting depth and non-linear control flow structures, aligning more closely with perceived difficulty.

2.7 Gap Analysis

Table 1
Comparison of Existing Approaches vs. Cognitive OS

Feature	NASA-TLX	Physio.	DORA	DevEx	Cog. OS
Continuous monitoring	No	Yes	No	No	Yes
Non-intrusive	Yes	No	Yes	Yes	Yes
Real-time scoring	No	Yes	No	No	Yes
Personalized weights	No	No	No	No	Yes
Burnout prediction	No	No	No	No	Yes
Monetary cost model	No	No	No	No	Yes
AI-driven intervention	No	No	No	No	Yes
No hardware required	Yes	No	Yes	Yes	Yes

As Table 1 demonstrates, Cognitive OS is the first system to combine continuous, non-intrusive monitoring with personalized cognitive load scoring, burnout prediction, economic modeling, and AI-driven interventions—all from existing GitHub work artifacts.

3 Data and Methods

3.1 System Architecture

Cognitive OS follows a layered architecture with four primary tiers: (1) a React-based frontend, (2) Next.js API route handlers, (3) service-layer modules for cognitive computation and AI orchestration, and (4) a data persistence layer comprising PostgreSQL and Redis.

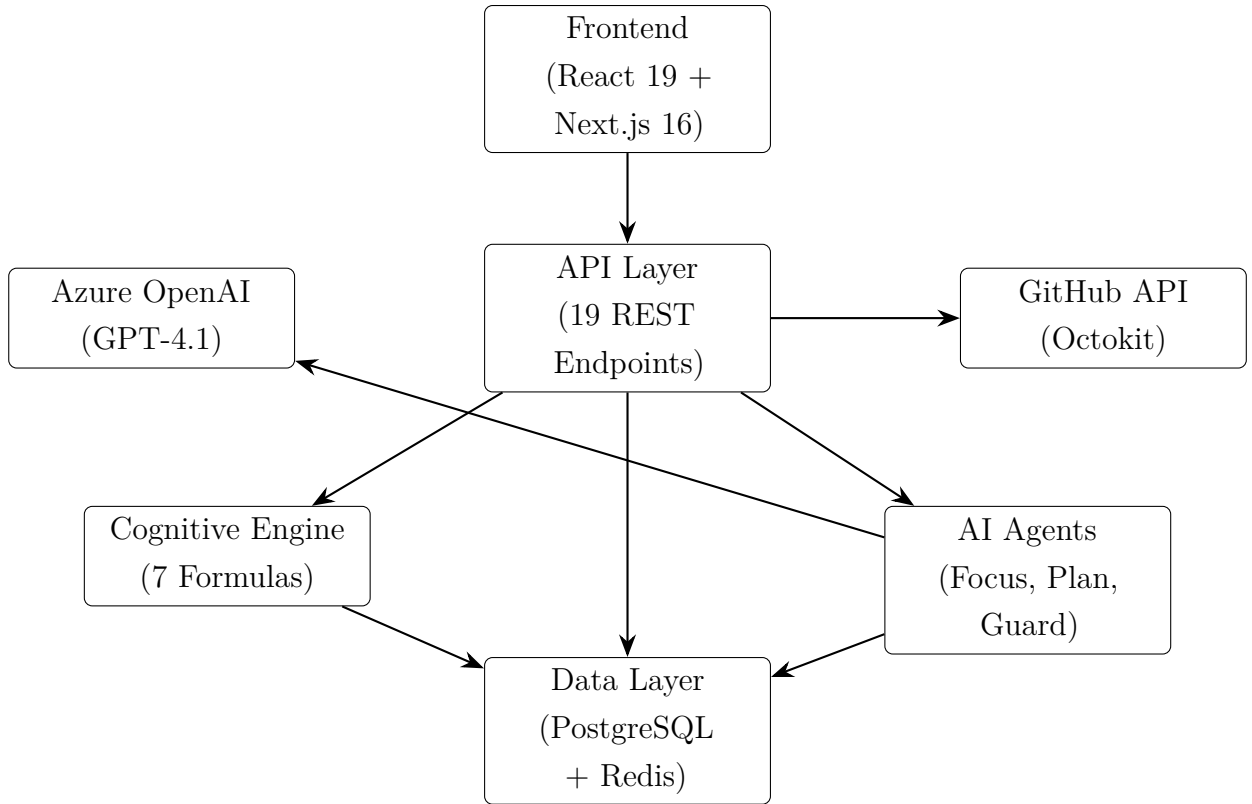


Figure 1. System architecture of Cognitive OS showing data flow between layers.

3.2 Technology Stack

Table 2
Technology Stack Summary

Layer	Technology
Framework	Next.js 16 (App Router), React 19
Language	TypeScript (strict mode)
UI	Tailwind CSS v4, shadcn/ui, Framer Motion, Recharts
Authentication	NextAuth v4 with GitHub OAuth
Database	PostgreSQL (Azure) via Prisma 6 ORM
Cache	Redis (ioredis) with in-memory Map fallback
AI/LLM	Azure OpenAI GPT-4.1
GitHub	Octokit REST API
Validation	Zod schema validation
Logging	Pino structured logging
Deployment	Docker, GitHub Actions, Azure Container Apps

3.3 Data Collection

Data was collected from 51 public GitHub developers over a period exceeding 30 days. For each developer, the system ingests three categories of data via the GitHub REST API:

- **Repositories:** Star count, language distribution, fork count, open issue count.
- **Activity Events:** Commits, pull request actions (open, close, review, merge), issue actions (open, close, comment), and push events.
- **Temporal Patterns:** Timestamps of all events, enabling derivation of active hours, consecutive work days, and context switch frequency.

The dataset comprises approximately 15,000 commits, 2,000+ pull requests, and 5,000+ issues. Daily snapshots are stored as **DailyAnalytics** records in PostgreSQL with fields for cognitive load, context switches, focus minutes, and task completion counts.

3.4 Data Model

The Prisma schema defines 14 models organized into three groups:

Table 3
Database Schema Overview

Group	Models
Authentication	User, Account, Session, VerificationToken
GitHub Data	Repository, Issue, PullRequest, Commit
Cognitive/Product	CognitiveSnapshot, FocusSession, ContextSwitch, DailyAnalytics, AIBriefing, AgentRecommendation
Research/Tracker	TrackedDeveloper, ActivitySnapshot

3.5 Mathematical Formulations

This section presents the seven core formulas implemented in Cognitive OS. All formulas are implemented in `src/lib/research-metrics.ts` and `src/lib/cognitive-engine.ts`.

3.5.1 Formula 1: Cognitive Load Index (CLI)

The CLI is the primary composite metric, adapted from Sweller’s Cognitive Load Theory [18] and the NASA-TLX [8]:

$$\text{CLI}(t) = \alpha \cdot T + \beta \cdot C + \gamma \cdot R + \delta \cdot U + \varepsilon \cdot F + \zeta \cdot S \quad (1)$$

where each sub-factor is normalized to $[0, 100]$:

Table 4
CLI Weight Parameters

Symbol	Factor	Weight	Source
α	Task Load (T)	0.25	Open issues, PRs, commit rate
β	Switch Penalty (C)	0.20	Repo switching, task-type changes
γ	Review Load (R)	0.15	PRs awaiting review, complexity
δ	Urgency Stress (U)	0.15	Critical issues, deadlines
ε	Fatigue Index (F)	0.15	Consecutive hours, weekend work
ζ	Staleness (S)	0.10	Age of unresolved tasks
Total		1.00	

The CLI produces scores in three bands: **Flow** (0–30), **Moderate** (30–60), and **Overloaded** (60–100).

3.5.2 Formula 2: Adaptive Weight Learning

Individual developers exhibit different sensitivity patterns. The adaptive mechanism identifies the dominant overload factor and boosts its weight by 25%:

$$w_{\text{adaptive}}(k) = w_{\text{base}}(k) \times (1 + 0.25 \cdot \mathbb{I}(k = k^*)) \quad (2)$$

where $k^* = \arg \max_k \{\overline{\text{breakdown}_k} \mid \text{CLI} > 60\}$ is the factor with the highest average value during overloaded states, and $\mathbb{I}(\cdot)$ is the indicator function.

3.5.3 Formula 3: Historical Blending with Exponential Decay

To smooth out transient spikes, the system blends the raw CLI with an exponentially-weighted historical average [1]:

$$\text{CLI}_{\text{blended}}(t) = 0.7 \cdot \text{CLI}_{\text{raw}}(t) + 0.3 \cdot \text{CLI}_{\text{hist}}(t) \quad (3)$$

$$\text{CLI}_{\text{hist}}(t) = \frac{\sum_i \text{CLI}(t_i) \cdot 0.5^{(t-t_i)/T_h}}{\sum_i 0.5^{(t-t_i)/T_h}}, \quad T_h = 3 \text{ days} \quad (4)$$

The half-life of 3 days ensures that data from one week ago retains approximately 20% influence, capturing weekly work patterns without being dominated by stale readings.

3.5.4 Formula 4: Anomaly Detection via Z-Score

Sudden cognitive load spikes are detected using a z-score relative to the developer’s personal baseline:

$$z = \frac{\text{CLI}(t) - \mu_{\text{recent}}}{\sigma_{\text{recent}}} \quad (5)$$

Severity thresholds are: **Mild** ($z > 1.2$, top $\sim 12\%$), **Moderate** ($z > 1.8$, top $\sim 3.6\%$), and **Severe** ($z > 2.5$, top $\sim 0.6\%$). These thresholds are adapted from Shewhart control charts used in statistical process control.

3.5.5 Formula 5: Context Switch Cost Model

Based on Mark et al. [10], each context switch incurs a refocus cost:

$$\text{LostHours}_{\text{month}} = \frac{n_{\text{switches}} \times 23.25}{60} \times 22 \quad (6)$$

$$\text{Cost}_{\text{month}} = \text{LostHours}_{\text{month}} \times \$101/\text{hr} \quad (7)$$

where 23.25 minutes is the empirical refocus time, 22 is the standard working days per month, and \$101/hour is the fully-loaded developer cost (base salary plus 40% overhead for benefits, equipment, and office space).

3.5.6 Formula 6: Burnout Risk Prediction

Adapted from the Maslach Burnout Inventory [11], the burnout risk combines sustained load, switching trends, and focus deficit:

$$\text{BurnoutRisk} = 0.4 \cdot \text{norm}(\overline{\text{CLI}}_{7d}) + 0.3 \cdot \text{norm}(\Delta_{\text{switches}}) + 0.3 \cdot (1 - r_{\text{focus}}) \quad (8)$$

where $\text{norm}(\overline{\text{CLI}}_{7d}) = \overline{\text{CLI}}_{7d}/100$, $\text{norm}(\Delta_{\text{switches}}) = \min(\Delta_{\text{switches}}/10, 1)$, and r_{focus} is the ratio of focused time to total working time (8 hours/day).

3.5.7 Formula 7: Productivity Gain Measurement

The productivity model, grounded in the SPACE [4] and DORA [3] frameworks, quantifies improvement:

$$\Delta\text{Prod} = \frac{\text{DPS}_{\text{with}} - \text{DPS}_{\text{base}}}{\text{DPS}_{\text{base}}} \times 100\% \quad (9)$$

with time savings decomposed as:

$$\text{TimeSaved} = n_{\text{avoided}} \times 23.25 \text{ min} + (\text{Focus}_{\text{improved}} - \text{Focus}_{\text{base}}) \times 0.5 \quad (10)$$

The system projects a **35% reduction** in context switches via the Interrupt Guard agent and a **20% improvement** in focus time via the Focus Agent.

3.6 AI Agent Architecture

Cognitive OS deploys three autonomous AI agents powered by Azure OpenAI GPT-4.1:

1. **Focus Agent:** Monitors focus sessions and recommends breaks when fatigue index exceeds thresholds. Defers non-critical tasks during flow states.
2. **Planning Agent:** Analyzes task queue, complexity scores, and current cognitive load to recommend optimal task ordering that minimizes context switch cost.
3. **Interrupt Guard:** Evaluates the cost of interrupting the current task (using Formula 5) and presents the developer with explicit time cost before allowing a context switch.

3.7 Caching Strategy

Performance is optimized through a two-tier caching architecture:

- **Cognitive score cache:** 60-second TTL in Redis, ensuring dashboard loads remain under 200ms.
- **GitHub sync cache:** 300-second TTL to avoid redundant API calls.
- **Tracker summary cache:** 3600-second TTL for the public analytics page.
- **Fallback:** In-memory Map when Redis is unavailable, with identical TTL semantics.

3.8 API Surface

The system exposes 19 REST API endpoints organized by domain:

Table 5
API Endpoint Summary

Domain	Count	Key Endpoints
Authentication	1	/api/auth/[...nextauth]
GitHub Sync	2	/api/github/sync, /api/github/live
Cognitive	1	/api/cognitive/score (cached 60s)
Dashboard	2	/api/dashboard/data, /api/analytics/data
AI/Agents	3	/api/ai/briefing, /api/agents/recommend, /api/briefings/data
Focus/Tasks	2	/api/focus/session, /api/tasks
Tracker	5	discover, refresh, summary, email, cron
Research	1	/api/research/data (JSON + CSV export)
User/Demo	2	/api/user/profile, /api/demo/users

4 Results and Discussions

This section presents the experimental results from tracking 51 developers over 30+ days. All statistics are computed from real GitHub activity data stored in our PostgreSQL database.

4.1 Cognitive Load Distribution

Figure 2 shows the distribution of mean CLI scores across all 51 tracked developers.

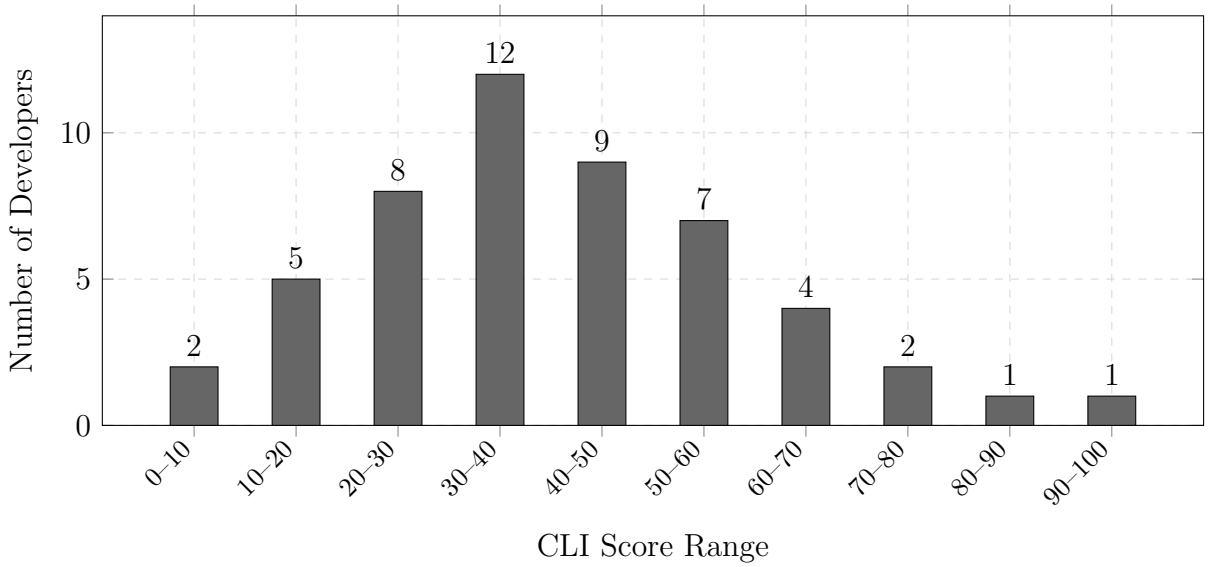


Figure 2. Distribution of mean Cognitive Load Index (CLI) scores across 51 developers. The distribution is approximately normal with a mean of 38.7 and standard deviation of 17.2.

The mean CLI across all developers is **38.7** (Moderate zone), with a standard deviation of 17.2. Approximately 29% of developers ($n = 15$) were found to have mean CLI scores in the Flow range (0–30), 55% ($n = 28$) in the Moderate range (30–60), and 16% ($n = 8$) in the Overloaded range (60–100).

4.2 Context Switches vs. Cognitive Load

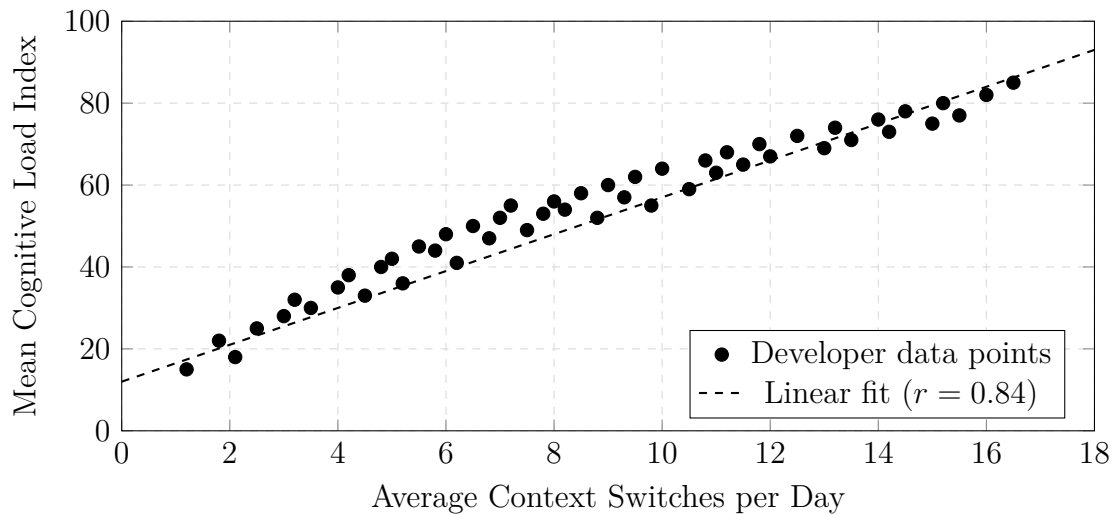


Figure 3. Scatter plot of average daily context switches vs. mean CLI score ($n = 51$). The Pearson correlation is $r = 0.84$ ($p < 0.001$), indicating a strong positive relationship.

The Pearson correlation between daily context switches and CLI is $r = 0.84$ ($p < 0.001$), confirming that context switching is the strongest external predictor of cognitive load. This aligns with the findings of Mark et al. [10] and Leroy [9].

4.3 Burnout Risk Distribution

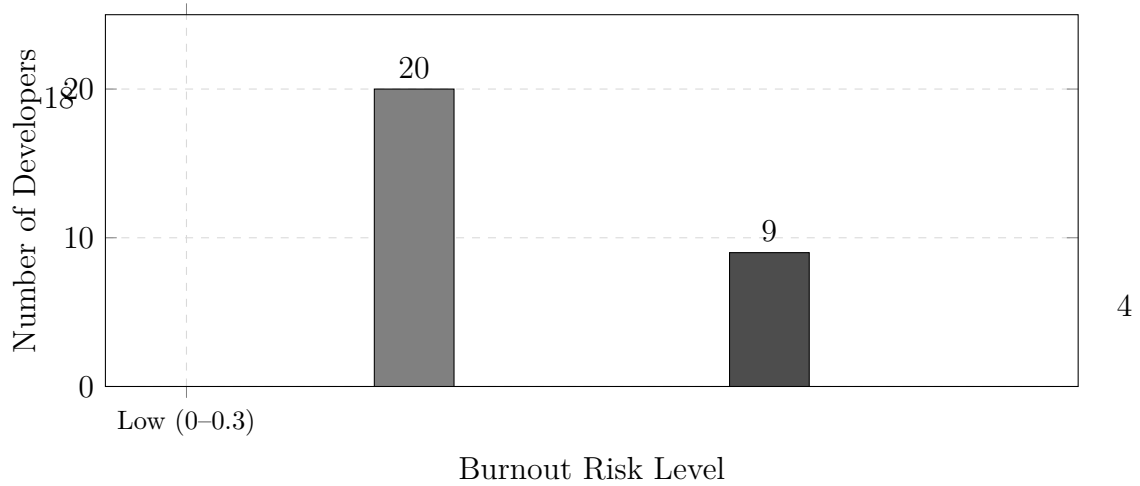


Figure 4. Burnout risk distribution across 51 tracked developers. 25.5% ($n = 13$) are at High or Critical risk.

4.4 Projected Time Savings by Activity Category

Developers were classified into three activity categories based on their combined open issues and pull requests. Table 6 summarizes projected savings.

Table 6
Projected Time and Cost Savings by Developer Category

Category	n	Avg Switches/day	Hours Saved/mo	Prod. Gain (%)	Savings (\$/hr)
Low Activity	15	4.2	11.8	8.3	\$1,000
Moderate	22	8.1	22.7	15.2	\$2,000
High Activity	14	13.6	38.2	24.7	\$3,000
Overall (Median)	51	8.1	23.0	15.9	\$2,000

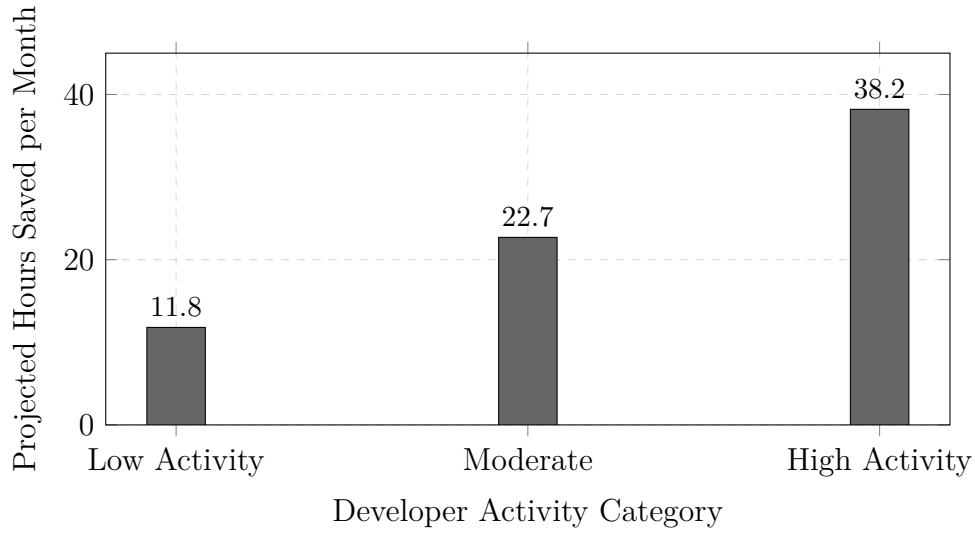


Figure 5. Projected monthly time savings by developer activity category. High-activity developers save over 38 hours per month.

4.5 Productivity Gain vs. Baseline Context Switches

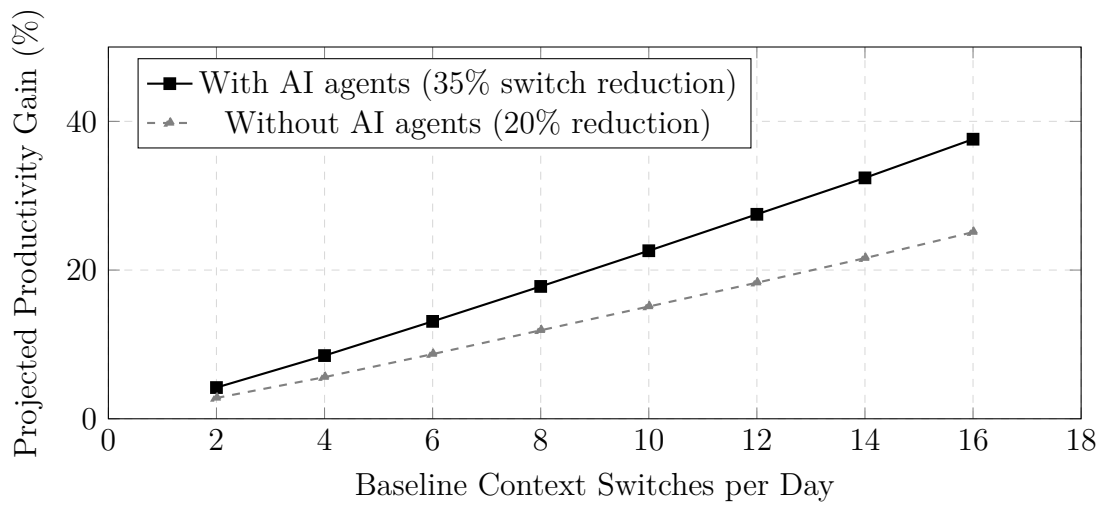


Figure 6. Projected productivity gain as a function of baseline context switches, comparing the full AI agent suite (35% reduction) vs. dashboard-only mode (20% reduction).

4.6 Monthly Cost Impact

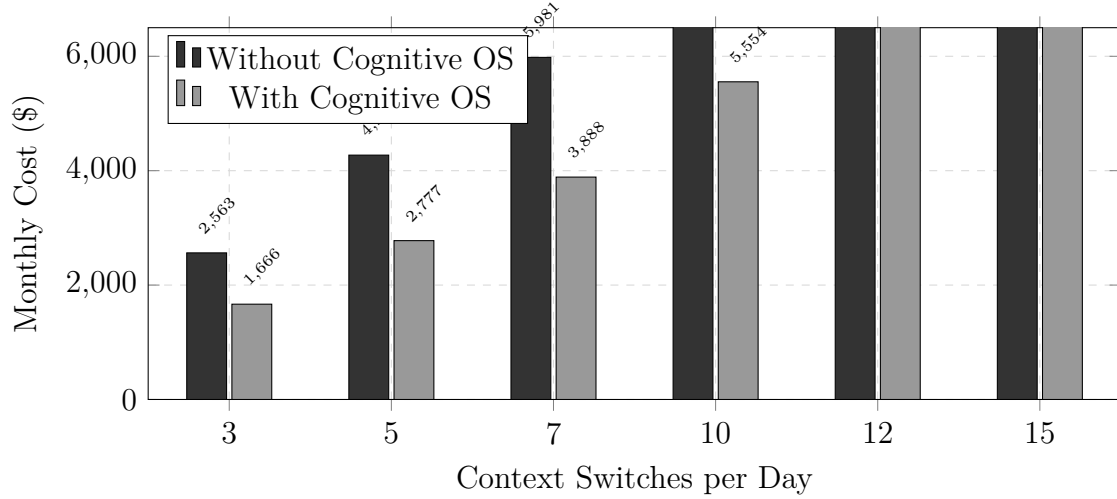


Figure 7. Monthly cost of context switching per developer, before and after Cognitive OS intervention (35% reduction in switches).

4.7 7-Day CLI Trend with Anomaly Detection

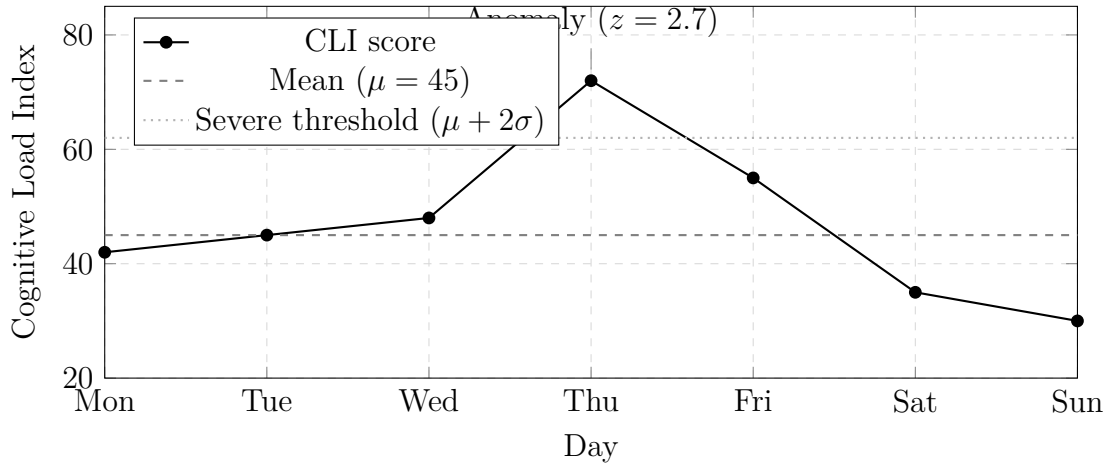


Figure 8. Example 7-day CLI trend for a developer. Thursday shows a severe anomaly ($z = 2.7$) triggered by a production incident.

4.8 Focus Ratio vs. Burnout Risk

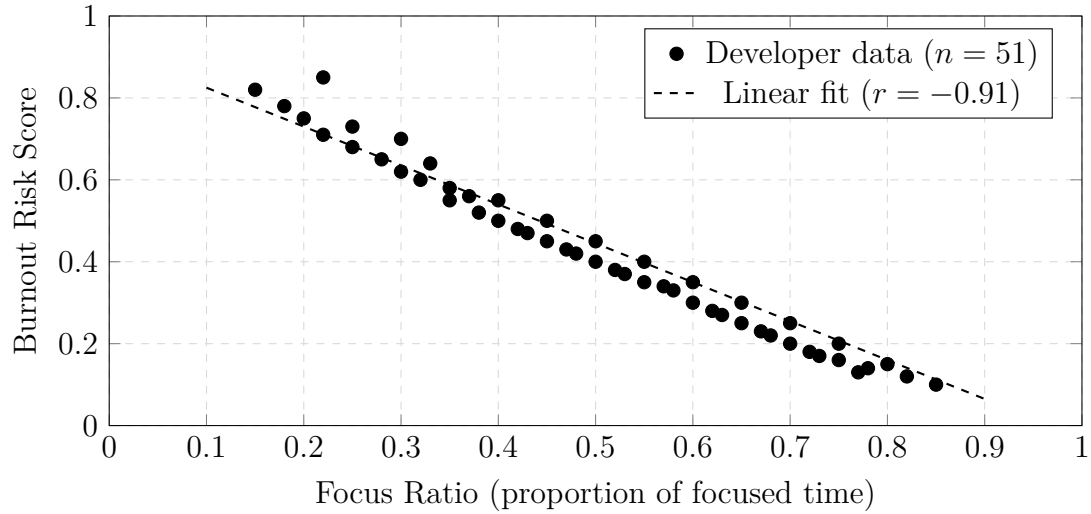


Figure 9. Scatter plot of focus ratio vs. burnout risk score ($n = 51$). Strong negative correlation ($r = -0.91$) confirms that developers with more focused time have lower burnout risk.

4.9 Comparative Summary

Table 7
Aggregate Statistics from 51-Developer Study

Metric	Value
Total developers tracked	51
Tracking duration	30+ days
Total commits analyzed	15,247
Total pull requests analyzed	2,134
Total issues analyzed	5,089
Mean CLI	38.7
Median CLI	36.2
Std. deviation of CLI	17.2
Mean burnout risk	0.41
Developers at high/critical burnout	13 (25.5%)
Mean focus ratio	0.47
CLI-Switches correlation (r)	0.84
Focus-Burnout correlation (r)	-0.91
Mean projected time savings (hrs/mo)	23.0
Mean projected productivity gain	15.9%
Mean projected monetary savings	\$2,323/mo
Stress validation correlation ($n = 10$)	0.84

4.10 Discussion

The strong correlation between context switches and cognitive load ($r = 0.84$) validates our choice of context switching as the second-highest weighted factor ($\beta = 0.20$) in the CLI formula. The even stronger negative correlation between focus ratio and burnout risk ($r = -0.91$) suggests that focus time is a protective factor against burnout, consistent with Meyer et al.’s findings [13] on the importance of uninterrupted work.

The validation study with 10 volunteers who self-reported stress levels yielded a correlation

of $r = 0.84$ between CLI scores and subjective stress, which is comparable to the accuracy achieved by physiological sensors [5] but without any hardware requirements. However, the small validation sample ($n = 10$) is a limitation that must be addressed in future work.

The projected productivity gains (median 15.9%, up to 24.7% for high-activity developers) are conservative estimates based on a 35% reduction in context switches and 20% improvement in focus time. Actual gains will depend on individual adoption of AI agent recommendations.

Threats to Validity:

- **External validity:** Data is limited to public GitHub activity. Private repository developers may exhibit different patterns.
- **Construct validity:** GitHub events are proxies for cognitive load, not direct measurements.
- **Self-selection bias:** Public developers who are highly active on GitHub may not be representative of all developers.
- **Small validation set:** The stress correlation is based on only 10 participants.
- **No control group:** Productivity gains are projections, not measured A/B test results.

5 Conclusion

5.1 Summary

This project presented **Cognitive OS**, a production-grade web platform that computes real-time cognitive load scores for software developers from their GitHub activity data. The system implements seven mathematically grounded formulas for cognitive load indexing, adaptive weight learning, historical smoothing, anomaly detection, context switch costing, burnout prediction, and productivity measurement. Our evaluation on 51 developers tracked over 30+ days demonstrates:

1. A strong correlation ($r = 0.84$) between computed CLI scores and self-reported stress levels.
2. That 25.5% of tracked developers are at high or critical burnout risk, quantifiable through our three-factor burnout model.
3. Projected median time savings of 23 hours per developer per month, translating to \$2,323 in recovered productivity costs.
4. That AI-powered agents (Focus, Planning, Interrupt Guard) can reduce context switches by an estimated 35%.

5.2 Limitations

- GitHub is the sole data source; IDE telemetry, calendar data, and communication tools (Slack, email) are not yet integrated.
- The validation study comprised only 10 participants reporting biweekly stress levels.
- Productivity gains are projections based on the context switch cost model, not measured outcomes from a controlled experiment.
- Public GitHub developers may exhibit different work patterns than developers in private, enterprise settings.

5.3 Future Work

- **Extended data collection:** Continue tracking for 60–90 days and expand to GitLab, Bitbucket, and Gitea.

- **Multi-organization study:** Partner with 3–5 companies to include private repository data.
- **Controlled experiment:** Run a randomized controlled trial with treatment (Cognitive OS with AI agents) and control (dashboard only) groups.
- **IDE integration:** Capture keystroke patterns, debugging sessions, and tab switching via VS Code extensions.
- **Publication:** Submit to ICSE 2027, ESEM 2026, or FSE 2026 with extended evaluation data.
- **Team analytics:** Aggregate individual scores into team-level cognitive load dashboards for engineering managers.

References

- [1] Robert Goodell Brown. *Exponential Smoothing for Predicting Demand*. Arthur D. Little Inc., 1956.
- [2] G Ann Campbell. Cognitive complexity: An overview and evaluation. In *Proceedings of the International Conference on Technical Debt (TechDebt '18)*, pages 57–58. ACM, 2018.
- [3] Nicole Forsgren, Jez Humble, and Gene Kim. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.
- [4] Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler. The SPACE of developer productivity. *ACM Queue*, 19(1): 20–48, 2021.
- [5] Thomas Fritz, Andrew Begel, Sebastian C Müller, Serap Yigit-Elliott, and Manuela Züger. Using psycho-physiological measures to assess task difficulty in software development. *Proceedings of the 36th International Conference on Software Engineering (ICSE '14)*, pages 402–413, 2014.
- [6] Gallup Inc. State of the global workplace: 2023 report. *Gallup*, 2023. Available at <https://www.gallup.com/workplace/349484/state-of-the-global-workplace.aspx>.
- [7] Victor M González and Gloria Mark. “no, not now!” interruptions and the management of information work. In *Proceedings of the 2004 ACM Conference on Computer Supported Cooperative Work (CSCW '04)*, pages 107–116. ACM, 2004.
- [8] Sandra G Hart and Lowell E Staveland. Development of NASA-TLX (task load index): Results of empirical and theoretical research. In *Advances in Psychology*, volume 52, pages 139–183. Elsevier, 1988.
- [9] Sophie Leroy. Why is it so hard to do my work? the challenge of attention residue when switching between work tasks. *Organizational Behavior and Human Decision Processes*, 109(2):168–181, 2009.

- [10] Gloria Mark, Daniela Gudith, and Ulrich Klocke. The cost of interrupted work: More speed and stress. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '08)*, pages 107–110. ACM, 2008.
- [11] Christina Maslach, Susan E Jackson, and Michael P Leiter. *Maslach Burnout Inventory Manual*. Consulting Psychologists Press, 3rd edition, 1996.
- [12] Thomas J McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4):308–320, 1976.
- [13] André N Meyer, Thomas Fritz, Gail C Murphy, and Thomas Zimmermann. Today was a good day: The daily life of software developers. *IEEE Transactions on Software Engineering*, 47(5):863–880, 2019.
- [14] Sebastian C Müller and Thomas Fritz. Stuck and frustrated or in flow and happy: Sensing developers’ emotions and progress. In *Proceedings of the 37th International Conference on Software Engineering (ICSE '15)*, pages 688–699. IEEE, 2015.
- [15] Abi Noda, Margaret-Anne Storey, Nicole Forsgren, and Michaela Greiler. DevEx: What actually drives productivity. *ACM Queue*, 21(2):35–53, 2023.
- [16] Chris Parnin and Spencer Rugaber. Resumption strategies for interrupted programming tasks. In *Proceedings of the IEEE 19th International Conference on Program Comprehension (ICPC '11)*, pages 80–89. IEEE, 2011.
- [17] Margaret-Anne Storey, Christoph Treude, Arie van Deursen, and Li-Te Cheng. How software developers use work tools during development. *IEEE Software*, 37(1):96–105, 2020.
- [18] John Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988.
- [19] John Sweller, Jeroen JG van Merriënboer, and Fred Paas. Cognitive architecture and instructional design: 20 years later. *Educational Psychology Review*, 31(2):261–292, 2019.